

HTTP Client

HTTP Client

- 1. HTTP Client Overview
- 2. Core Concepts
 - 2.1 HTTPClient Request Flow
 - 2.2 HTTP Status Codes
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Code Analysis
 - 5.1 Configuration and Global Variables
 - 5.2 Send GET Request and Parse Response
- 6. Operation Process
 - 6.1 Build and Flash
 - 6.2 Operation Steps
 - 6.3 Verification Methods
- 7. FAQ

1. HTTP Client Overview

HTTP (HyperText Transfer Protocol) is the most widely used application layer protocol on the Internet. It is based on TCP and uses a **request-response** model: the client sends a GET request, and the server returns a status code + data. This experiment uses the `HTTPClient` library to send a GET request to `example.com`, parsing the HTTP status code and response body. From URL parsing, DNS lookup to TCP connection, sending request, and reading response, all are encapsulated by the library, done with just a few lines of code.

Typical Application Scenarios:

Application Type	Example
Calling REST API	Get weather, query database
Data Reporting	Sensor data uploaded to cloud platform via HTTP POST
Firmware Download	OTA downloads .bin firmware from server
Web Scraping	Crawl simple JSON information

2. Core Concepts

2.1 HTTPClient Request Flow

For a single HTTP GET request, the `HTTPClient` library automatically completes 4 stages:

```

http.begin(url)          establish connection
    ↓    URL parsing → DNS resolution → TCP three-way handshake (blocking)
http.GET()              send request
    ↓    construct HTTP request line + headers → send → wait for response
    ↓    return HTTP status code (int), positive = success, negative = network error
http.getString()        read response body
    ↓    read response body into String (small data) or getStream() streaming read (large
data)
http.end()              close connection
                    TCP four-way handshake, release socket, must be called

```

Method	Function	Blocking?
<code>http.begin(url)</code>	URL parsing + DNS + TCP connection	Yes
<code>http.GET()</code>	Send GET request, return status code	Yes (wait for response)
<code>http.POST(data)</code>	Send POST request with form body	Yes (wait for response)
<code>http.addHeader(key, val)</code>	Add custom request header before GET/POST	No
<code>http.getString()</code>	Read full response body as String	Yes
<code>http.getStream()</code>	Get streaming response body (large file/JSON stream)	No (streaming read)
<code>http.end()</code>	Close TCP connection, release resources	No

2.2 HTTP Status Codes

Return value logic for `http.GET()` :

```

> 0    HTTP status code (200/404/500, etc.), request reached server
= 0    No valid HTTP response received
< 0    Network layer error (-1 = connection failed, -11 = timeout)

```

Status Code	Meaning	Handling in This Experiment
200 OK	Request successful	Print response body
301 Moved Permanently	Permanent redirect	—
404 Not Found	Resource not found	—
500 Internal Server Error	Server internal error	—

Status Code	Meaning	Handling in This Experiment
-1	Connection failed (DNS failed/unreachable)	Print error message
-11	Request timeout	Print error message

3. Hardware Dependencies

Pure software experiment. Must first connect to router via WIFI-STA and ensure internet access (example.com).

4. Project Architecture and Flow

```

setup()
├ connectWiFi()
├ http.begin(URL)           // parse URL → DNS → TCP connection
├ code = http.GET()         // send GET request
├ if code == 200:
│   └ payload = http.getString() // read response body
└ http.end()                // close TCP connection

```

5. Code Analysis

Complete source code: `Source code → Arduino → 3.Network Communication → HTTP_Client → HTTP_Client.ino`

5.1 Configuration and Global Variables

```

#include <HTTPClient.h>
const char* GET_URL = "http://example.com/get";

```

5.2 Send GET Request and Parse Response

```

void setup() {
  Serial.begin(115200);
  connectWiFi();

  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;

    // begin() internally: parse URL → DNS resolution → TCP connection
    http.begin(GET_URL);

    // GET() returns HTTP status code, negative = network error
    int httpCode = http.GET();
  }
}

```

```
if (httpCode > 0) {  
    Serial.printf("HTTP Code: %d\n", httpCode);  
    if (httpCode == HTTP_CODE_OK) {    // 200  
        String payload = http.getString();  
        Serial.printf("Response: %s\n", payload.c_str());  
    }  
} else {  
    Serial.printf("Request failed: %s\n", http.errorToString(httpCode).c_str());  
}  
http.end();    // close TCP connection, must be called  
}
```

`http.begin(url)` internally completes URL parsing + DNS resolution + TCP three-way handshake;

`http.end()` sends FIN to close connection. For POST requests, use `http.addHeader()` to add headers and `http.POST(data)` to send form body.

6. Operation Process

6.1 Build and Flash

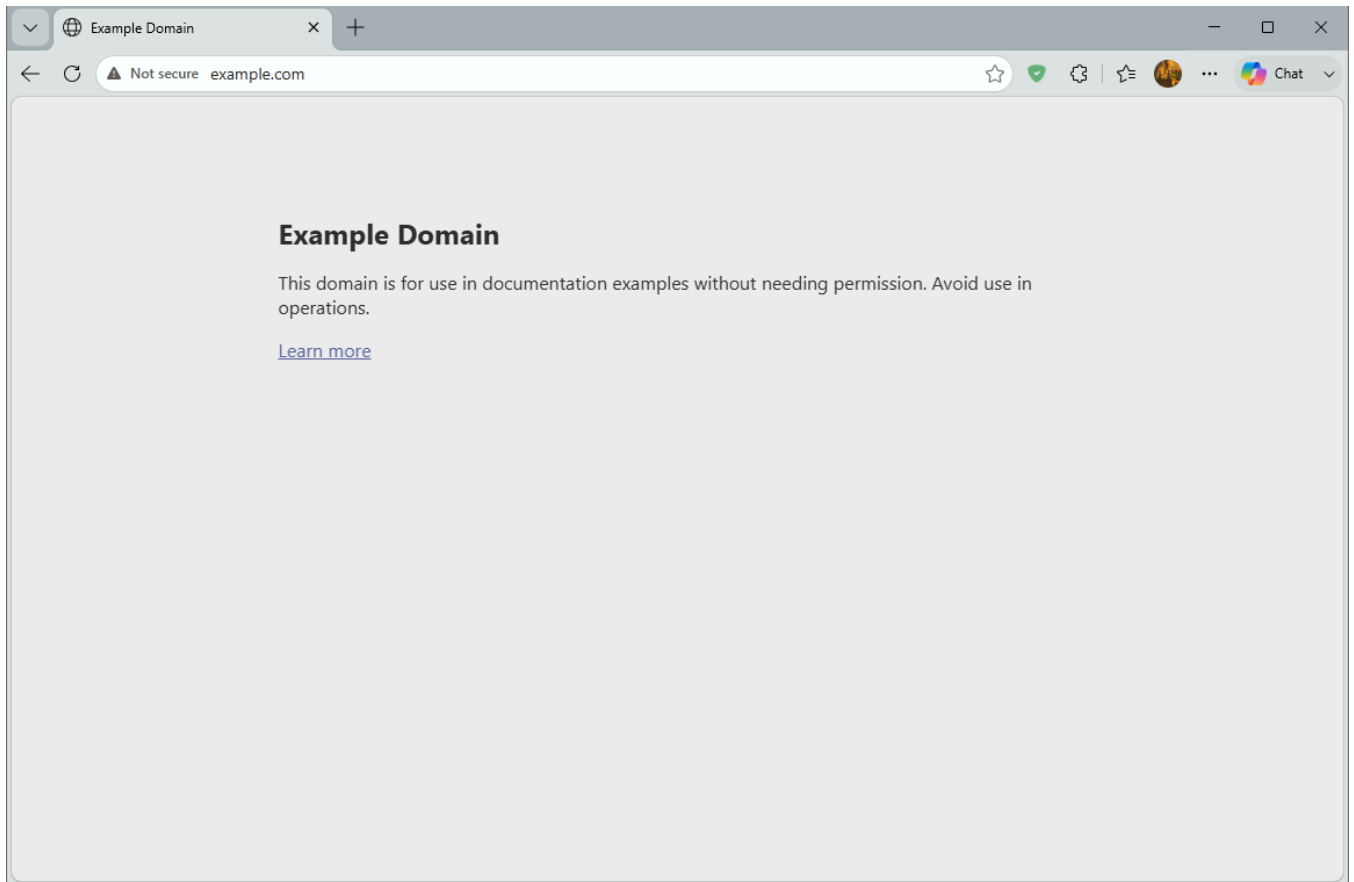
Arduino flashing process: See `Arduino → 1. Before Development → 2. Build and Flash`.

1. Ensure router can access internet (example.com)
2. Build and flash

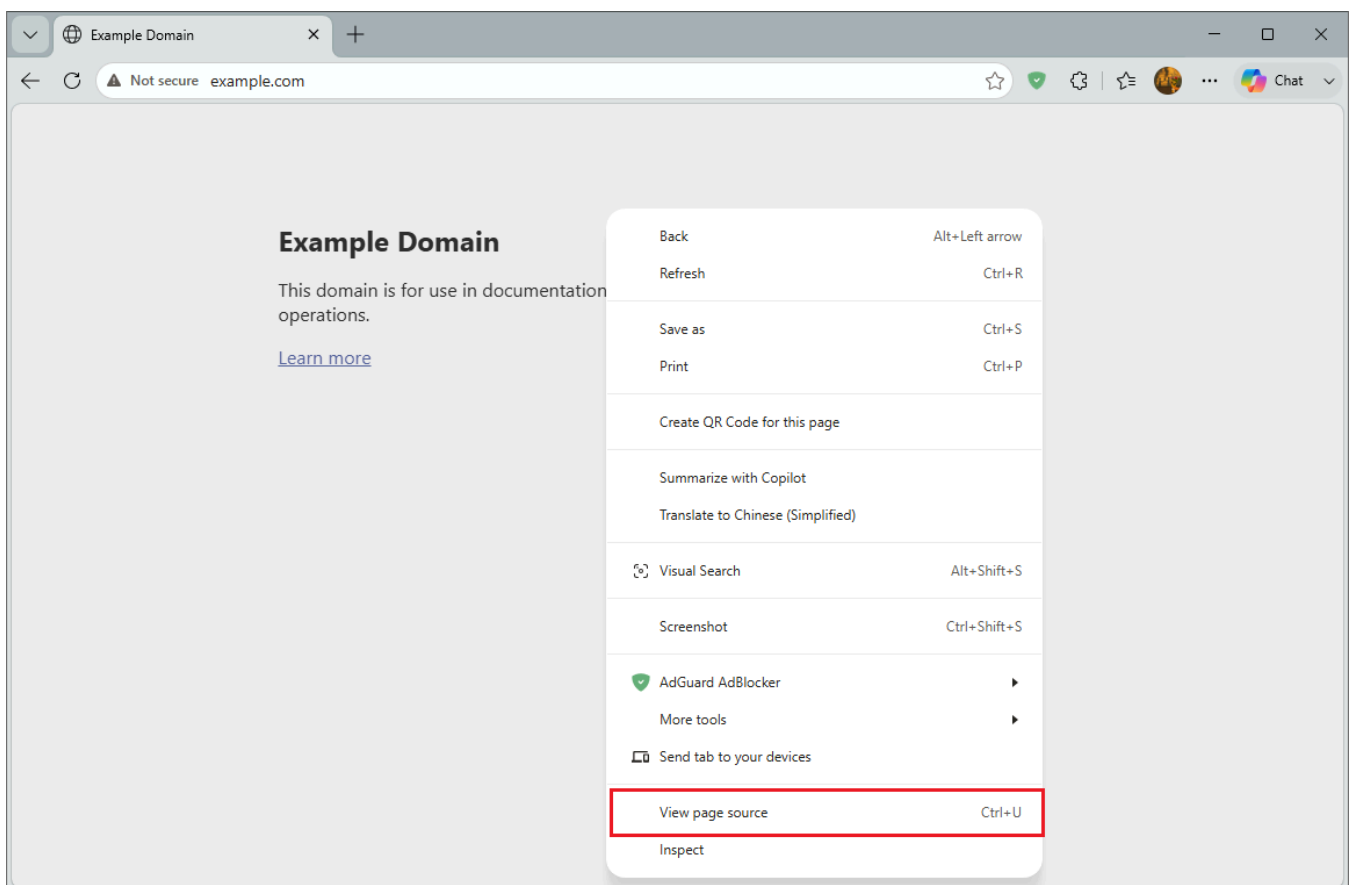
6.2 Operation Steps

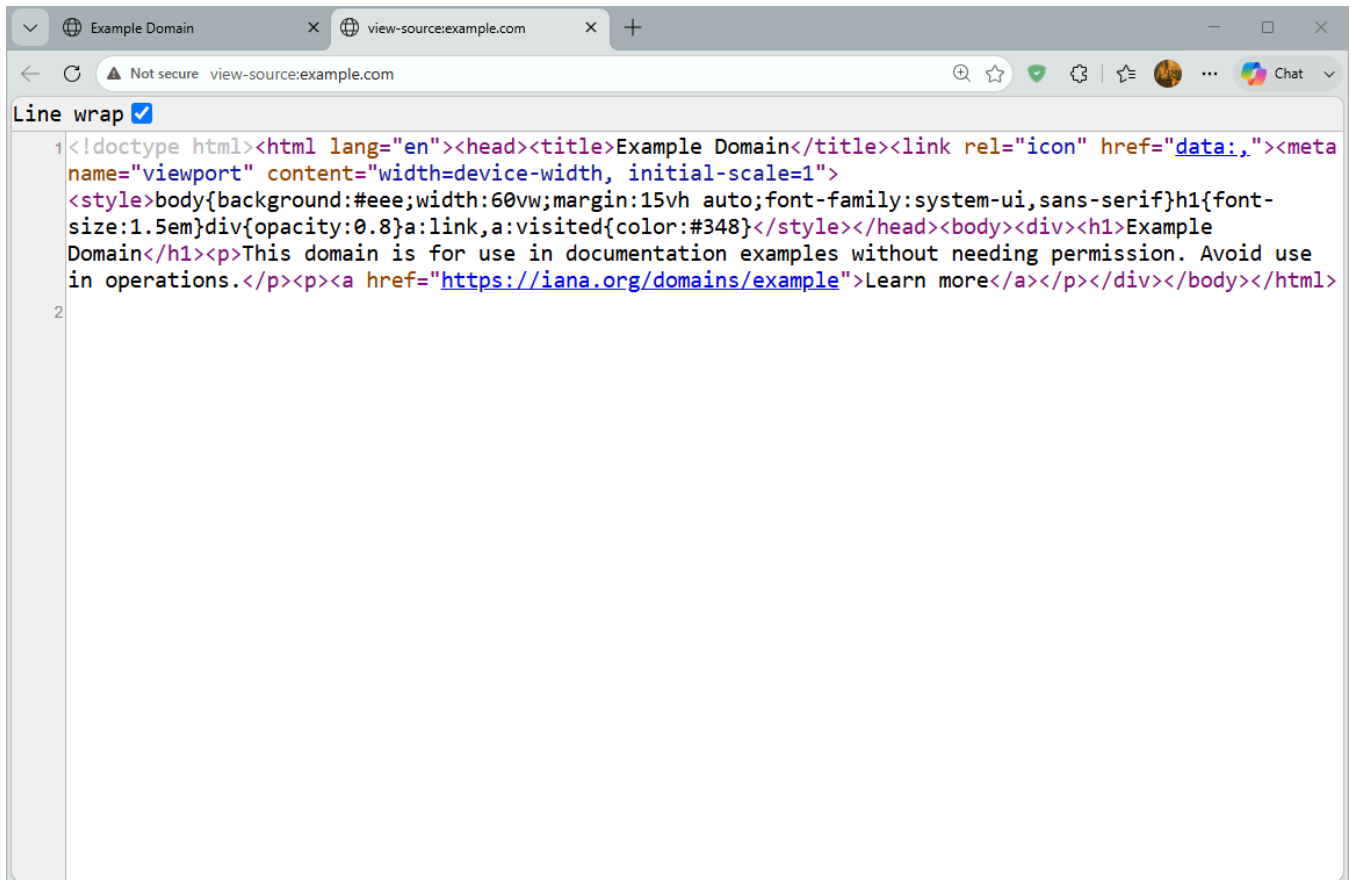
Step 1: Confirm Network Environment

Ensure the router connected to PC and ESP32 can access the internet. Open `http://example.com` in PC browser, should see the example webpage:



Right-click page and select "View Page Source", can see the returned HTML content:





```
1<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta
name="viewport" content="width=device-width, initial-scale=1">
<style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-
size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example
Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use
in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
2
```

Step 2: Build, Flash, and Open Serial Monitor

Build and flash the code to ESP32-S3 development board. Open serial monitor (baud rate 115200), wait for WiFi connection success.

Step 3: Observe HTTP Request Results

After ESP32 connects to WiFi, it automatically sends a GET request to `http://example.com/`. Serial monitor prints HTTP status code and HTML response from server:

```
HTTP Code: 200
Response: <!doctype html><html>...
```

```
WiFi Connected successfully!
Device IP: 192.168.11.181
Target Server: http://example.com
HTTP Code: 200
Response: <!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width
```

- `HTTP Code: 200` means request successful
- `Response` prints the complete HTML page source returned by example.com

Step 4: Verify Error Handling

Disconnect router from internet (unplug WAN cable, keep LAN WiFi normal), press RST to restart ESP32. At this point DNS resolution or TCP connection fails, `GET()` returns negative error code, serial prints error message like `Request failed: connection refused`.

6.3 Verification Methods

Operation	Expected Result
Normal internet	HTTP 200, print JSON response
Disconnect cable	<code>begin()</code> fails or <code>GET()</code> returns negative

7. FAQ

Phenomenon	Cause	Solution
<code>GET()</code> returns -1	DNS failed or target unreachable	Check if you can ping example.com
<code>getString()</code> is empty	Response body too large or Content-Length=0	Use <code>getStream()</code> for large files streaming
Garbled Chinese	Terminal encoding mismatch	Use UTF-8 terminal